

Collectives

Lecture 12

February 16, 2026

MPI_Comm_split Example Code Recap

Output
for P=8

```
int newrank, newsize, color = myrank%2;
MPI_Comm newcomm;
MPI_Comm_split (MPI_COMM_WORLD, color, myrank,
&newcomm);
MPI_Comm_rank (newcomm, &newrank);
MPI_Comm_size (newcomm, &newsize);
printf ("%d: %d of %d\n", myrank, newrank,
newsize);
MPI_Comm_free (&newcomm);
```

0: 0 of 4

1: 0 of 4

2: 1 of 4

3: 1 of 4

4: 2 of 4

5: 2 of 4

6: 3 of 4

7: 3 of 4

Student Doubt

Query refers to slide 6 of lecture 11

it is mentioned at the bottom of the slide : 2 sub-communicators suffice: split 1 (blue and orange processes) and split 2 (orange rectangles for each of the four sub-groups)

Just to be precise with the numbers are there 2 sub-communicators or $5 - 1$ for splitting between the blue and orange ranks and one each for clubbing together the 4 producer groups with their respective consumer.

This is because when I will use `MPI_Comm_split`, as per the mpich documentation (https://www.mpich.org/static/docs/v3.3/www3/MPI_Comm_split.html) Processes with the same color are in the same new communicator and I will be using 4 colors to ensure these 4 different "contexts of communication.

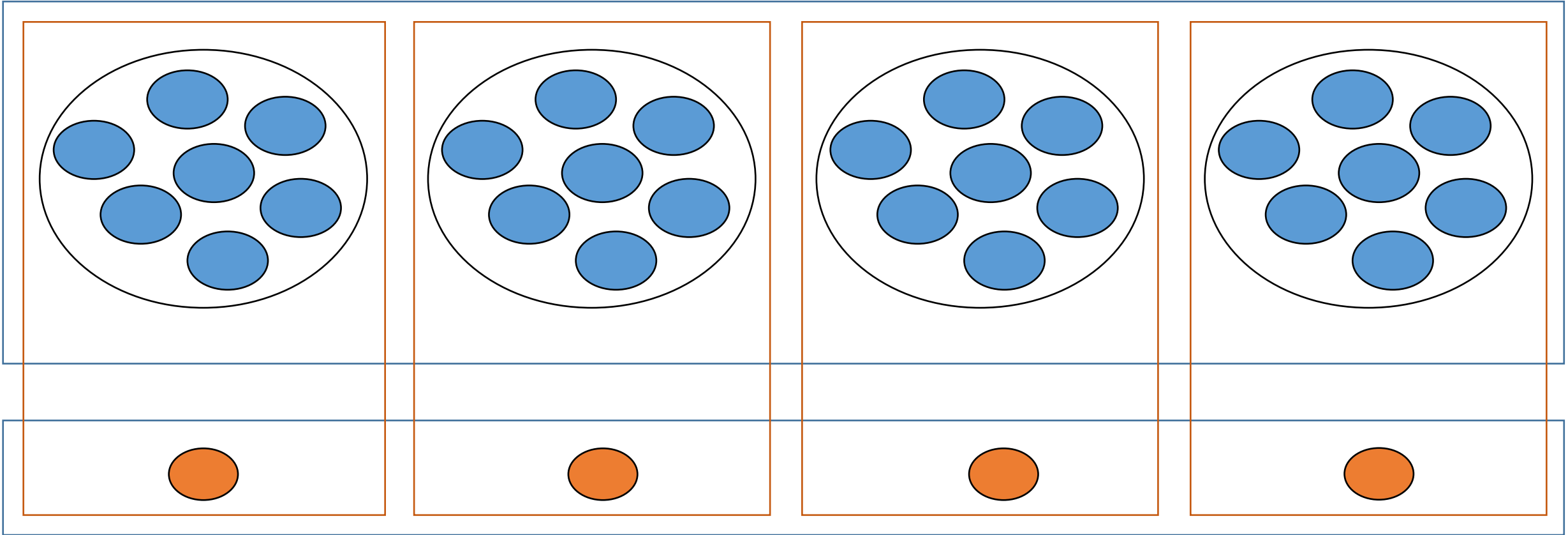
please correct me if i am wrong with the numbers here

Also suppose that the blue ranks from one of the contexts do not communicate anything with the blue ranks from another context, then we will not need the first split right? only the last one grouping together the blue ones with the respective orange ones should suffice?

A: Every process is part of two sub-groups. For this, we do not need 4 sub-communicators: refer to Lecture 11: slide 14 example

Producer-Consumer Example Recap

An example configuration: 28 simulation processes and 4 analysis processes.
Two splits required: (1) horizontal split and (2) vertical split



2 communication contexts: 1 for intra-blue or intra-orange process communications, another for the blue->orange producer to consumer process communications.

Sub-communicator

0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31

rows X cols 2D virtual process topology

- (1) Split into blue and orange groups
- (2) Split row-wise, i.e. form “rows”
number of subgroups

```
int color1 =  
myrank%cols<cols-1?1:0;  
MPI_Comm_split  
(MPI_COMM_WORLD, color1,  
myrank, &newcomm1);
```

```
int color2 = myrank/cols;  
MPI_Comm_split  
(MPI_COMM_WORLD, color2,  
myrank, &newcomm2);
```

Sub-communicator Example

0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31

4 x 8 2D virtual process topology

Homework: Every process computes local array statistics (sum/average). Select 1 process per row to write row-wise cumulative statistics to a file.

Input: 2D topology (rows, cols)

Code

Homework: Every process computes local array statistics (sum/average). Select 1 process per row to write row-wise cumulative statistics (may be one or more values) to a file.

Input: 2D topology (rows, cols)

0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31

4 x 8 2D virtual process topology

```
int color = myrank/cols;
MPI_Comm_split
(MPI_COMM_WORLD, color,
myrank, &newcomm);

MPI_Gather(... , 0, newcomm);
//All newrank 0 will write
```

```
int MPI_Gather (sendbuf, sendcount, sendtype,
recvbuf, recvcount, recvtype, root, comm)
```

Example – Gather at 0

```
int MPI_Gather (sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root, comm)
```

0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31

Input: rows, cols

column-wise sub-comms

MPI_Comm_split (... , color=myrank%cols, ...)

or

MPI_Comm_split (... , color=myrank/cols, ...)

row-wise sub-comms

//newrank | newsize | newcomm

MPI_Gather (...., 0, ... newcomm)

Next? (depends on comm_split)

MPI_Group_incl (...)

MPI_Comm_create_group

(comm, group, tag, collcomm)

Collective Communication

- Must be called by all processes that are part of the communicator

Types

- Synchronization (MPI_Barrier)
- Global communication (MPI_Bcast, MPI_Gather, ...)
- Global reduction (MPI_Reduce, ..)

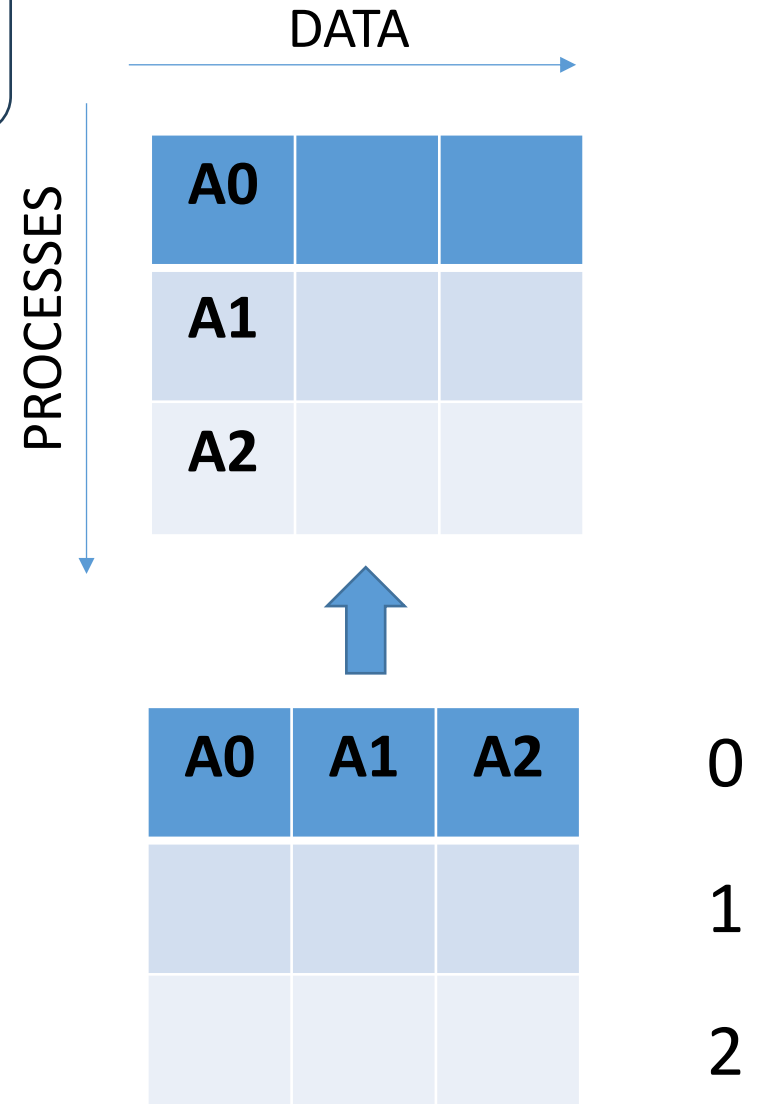
Blocking Collectives

- MPI_Barrier
- MPI_Bcast
- MPI_Gather
- MPI_Scatter
- MPI_Reduce

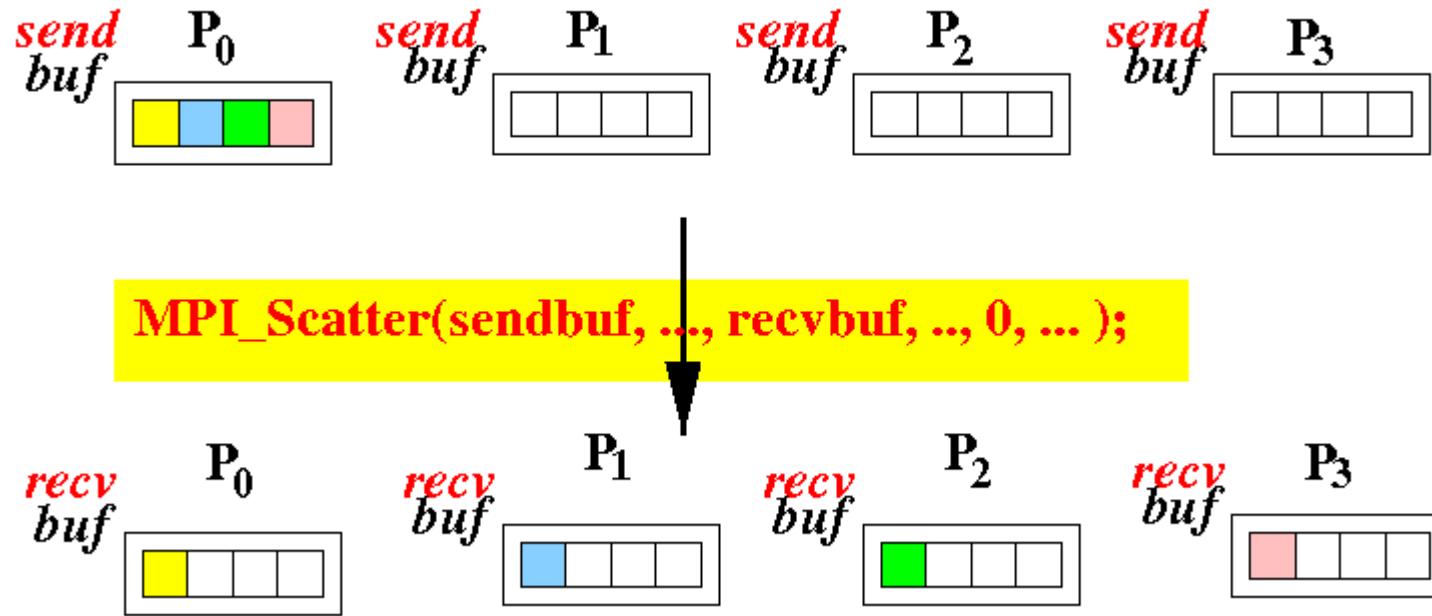
Scatter

- Scatters values to all processes from a root process
- `int MPI_Scatter (sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root, comm)`
- Arguments `send*` irrelevant in non-root processes
- `sendcount` – Number of elements sent to each process
- Output parameter – `recvbuf`

*count are an integer variables (similar to send/recv calls)



MPI_Scatter Illustration



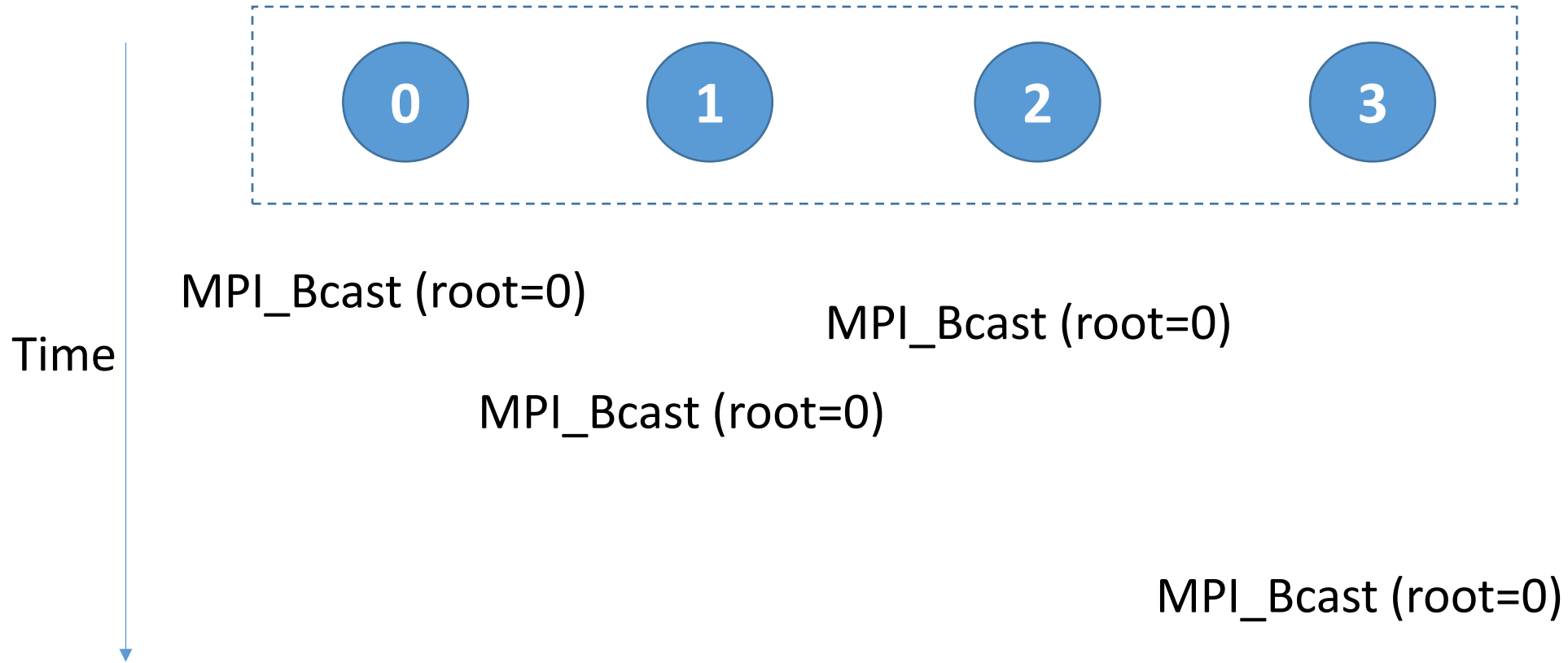
Bcast and Scatter Example

Q: Process 0 reads a file for t iterations (assume some streaming data being written to the file). The file size may vary across runs. Process 0 has to scatter the file content to all processes. How do we achieve this?

Buffer size of "buffer" array is not known apriori at non-root processes, how should root scatter the buffer?

A: Use `MPI_Bcast` to broadcast the size to all processes, only then the processes can use `MPI_Scatter` (with the correct counts).

Timing



All processes may not start and end at the same time.

Reduce

`MPI_Reduce` (*inbuf*, *outbuf*, count, datatype, op, root, comm)

- Combines element in *inbuf* of each process
- Combined value in *outbuf* of root
- op: MIN, MAX, SUM, PROD, ...
- Example:

`MPI_Reduce (... , 1, MPI_INT, MPI_MAX, ...)` Output: 21

`MPI_Reduce (... , 1, MPI_INT, MPI_MIN, ...)` Output: 1

21

5

1

8

3

2

13

Reduce

`MPI_Reduce` (*inbuf*, *outbuf*, count, datatype, op, root, comm)

`MPI_Reduce (... , 2, MPI_INT, MPI_MAX, ...)`

`MPI_Reduce (... , 2, MPI_INT, MPI_SUM, ...)`

21	1
5	15
1	4
8	81
3	30
2	22
13	33

Reduce

MPI_Reduce (*inbuf*, *outbuf*, count, datatype, op, root, comm)

MPI_Reduce (... , 1, MPI_FLOAT, MPI_PROD, 0, ...)

MPI_Reduce (... , 1, MPI_FLOAT, MPI_SUM, 0, ...)

MPI_Reduce (... , 1, MPI_FLOAT, MPI_PROD, 5, ...)

MPI_Reduce (... , 1, MPI_FLOAT, MPI_SUM, 5, ...)

2.6

5.7

1.4

8.2

3.4

2.2

1.3

Reduce

```
MPI_Reduce(&flnum, &val_p, 1, MPI_FLOAT, MPI_PROD, 0, MPI_COMM_WORLD);  
MPI_Reduce(&flnum, &val_p, 1, MPI_FLOAT, MPI_PROD, 5, MPI_COMM_WORLD);  
  
if (myrank == 0 || myrank == 5)  
    printf ("%d: %f %f\n", myrank, val_s, val_p);
```

260855.593750

260855.609375

Optional reading: What Every Computer Scientist Should Know About Floating-Point Arithmetic, by David Goldberg, published in the March, 1991 issue of Computing Surveys.

Data Input Example – File read by 0

0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31

4 x 8 2D virtual process topology

Scenario 1: Rank 0 scatters to all processes

Scenario 2: Rank 0 scatters to row or column leaders, which in turn scatters to their respective rows or columns

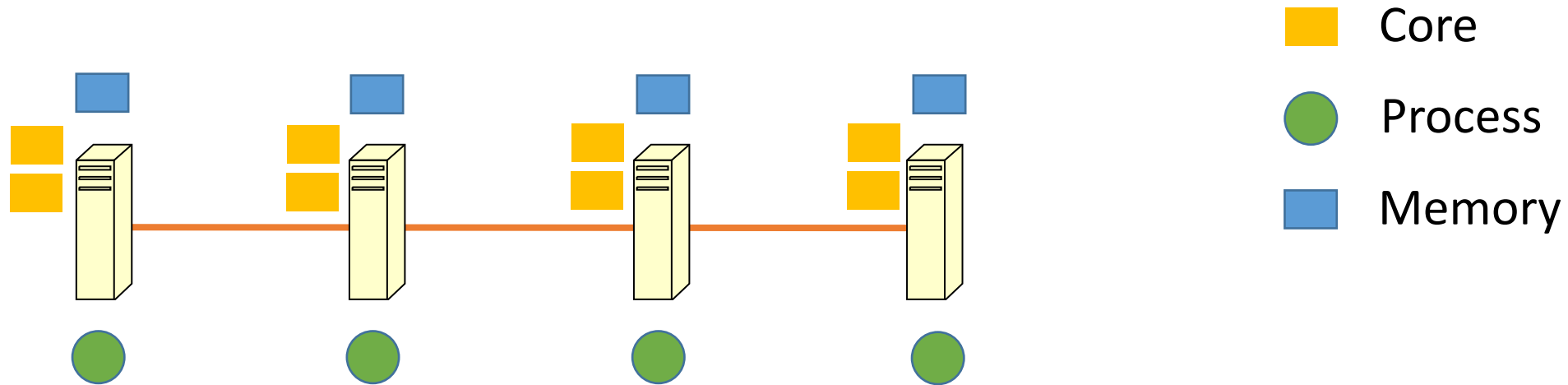
```
if (rank == 0)
    read data
```

```
// bcast count per process
```

```
// scatter (use the above count)
```

```
int MPI_Scatter (sendbuf, sendcount,
sendtype, recvbuf, recvcount, recvtype,
root, comm)
```

Parallel Sum



for $i = 0$ to N/P $\text{sum} += a[i] * a[i]$	for $i = N/P$ to $2N/P$ $\text{sum} += a[i] * a[i]$	for $i = 2N/P$ to $3N/P$ $\text{sum} += a[i] * a[i]$	for $i = 3N/P$ to N $\text{sum} += a[i] * a[i]$
---	--	---	--

Rank = 0

Rank = 1

Rank = 2

Rank = 3

Alternately:
Process 0 can scatter
the required part of
the array

Using Send/Recv to Collect the Partial Sums

```
int recvarr[numtasks];

// receive partial sums at rank 0
stime = MPI_Wtime();
if (rank)
{
    MPI_Send(&sum, 1, MPI_INT, 0, rank, MPI_COMM_WORLD);
}
else
{
    for (int r=1; r<numtasks; r++)
    {
        MPI_Recv(&recvarr[r], 1, MPI_INT, r, r, MPI_COMM_WORLD, &status);
    }
}
etime = MPI_Wtime();
cotime = etime - stime;

stime = MPI_Wtime();
// HOMEWORK
// Add the partial sums
for (int r=1; r<numtasks; r++)
    sum += recvarr[r];
etime = MPI_Wtime();
ctime += etime - stime;
```

Parallel Sum Alternate Code

```
// local computation at every process
for i = N/P * rank ; i < N/P * (rank+1) ; i++
    localsum += a[i] * a[i]

// collect localsum, add up at one of the ranks
MPI_Reduce (&localsum, ... , MPI_SUM, ...)
```

Using Reduce to Collect the Partial Sums

```
// local sum computation
sum=0.0;
stime = MPI_Wtime();
for (i=sidx; i<sidx+N/numtasks ; i++)
    sum += a[i] * a[i];
etime = MPI_Wtime();
ctime = etime - stime;

int globalsum;

stime = MPI_Wtime();
MPI_Reduce (&sum, &globalsum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
etime = MPI_Wtime();
cotime = etime - stime;

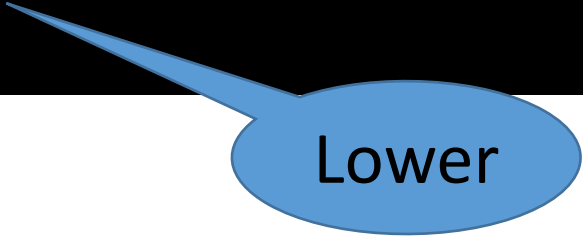
if (!rank)
printf ("%d %lf %lf\n", globalsum, ctime, cotime);
```

Timing

```
class $ for i in `seq 1 5`; do mpirun -np 3 ./parsum 6000 ; done
10000 0.000007 0.000301
10000 0.000006 0.000309
10000 0.000010 0.000015
10000 0.000006 0.000281
10000 0.000010 0.000041
class $ for i in `seq 1 5`; do mpirun -np 3 ./parsumreduce 6000 ; done
10000 0.000011 0.000172
10000 0.000010 0.000034
10000 0.000010 0.000033
10000 0.000010 0.000025
10000 0.000010 0.000028
```

Timing

```
class $ for i in `seq 1 5`; do mpirun -np 30 -hosts csews3:10,csews5:10,csews6:10 ./parsum 6000000 ; done
1711000000 0.001006 0.025789
1711000000 0.001034 0.010492
1711000000 0.001007 0.023931
1711000000 0.001029 0.038722
1711000000 0.001028 0.024971
class $ for i in `seq 1 5`; do mpirun -np 30 -hosts csews3:10,csews5:10,csews6:10 ./parsumreduce 6000000 ; done
1711000000 0.001003 0.011949
1711000000 0.002717 0.001540
1711000000 0.001092 0.009885
1711000000 0.003251 0.003160
1711000000 0.001091 0.012467
```



Lower

An efficient algorithm is implemented for MPI_Reduce, which is better than naïve many-to-one followed by summation.

Blocking Collectives

- MPI_Barrier
- MPI_Bcast
- MPI_Gather
- MPI_Scatter
- MPI_Reduce
- MPI_Allgather
- MPI_Allreduce
- MPI_Alltoall

Allgather

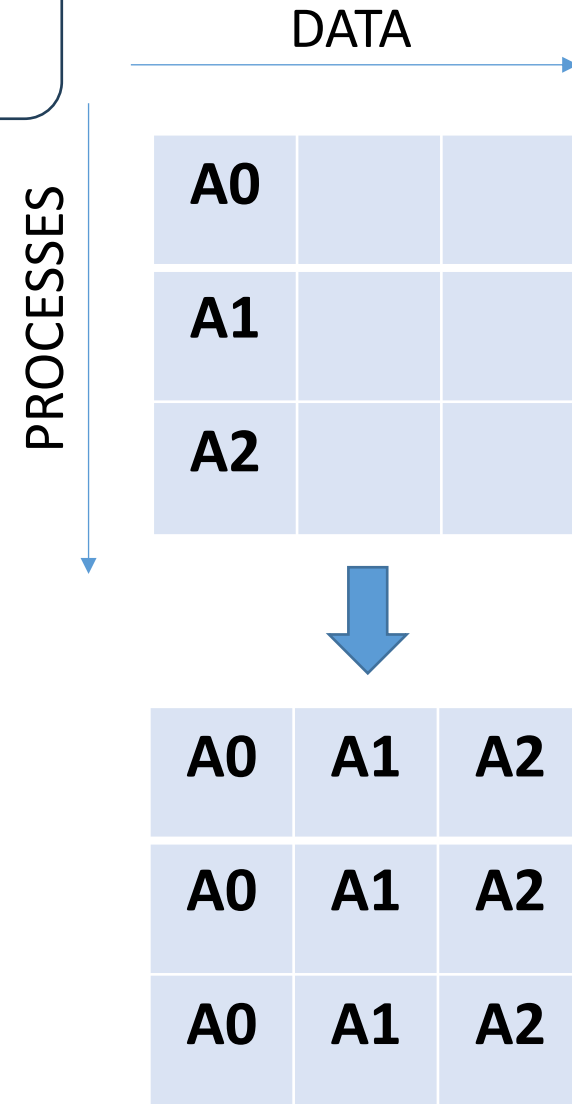
*count are an integer variables (similar to send/recv calls)

- All processes gather values from all processes
- `int MPI_Allgather (sendbuf, sendcount, sendtype, recvbuf, recvcount, recvttype, comm)`
- No root process

Example code

```
recvbuf = (int *) malloc  
(size*100*sizeof(int));
```

```
MPI_Allgather (sendbuf, 100, MPI_INT,  
recvbuf, 100, MPI_INT, comm);
```



Allreduce

- `MPI_Allreduce` (inbuf, *outbuf*, count, datatype, op, comm)
- op: MIN, MAX, SUM, PROD, ...
- Combines element in inbuf of each process
- Combined value in outbuf of each process

Equivalent collective?

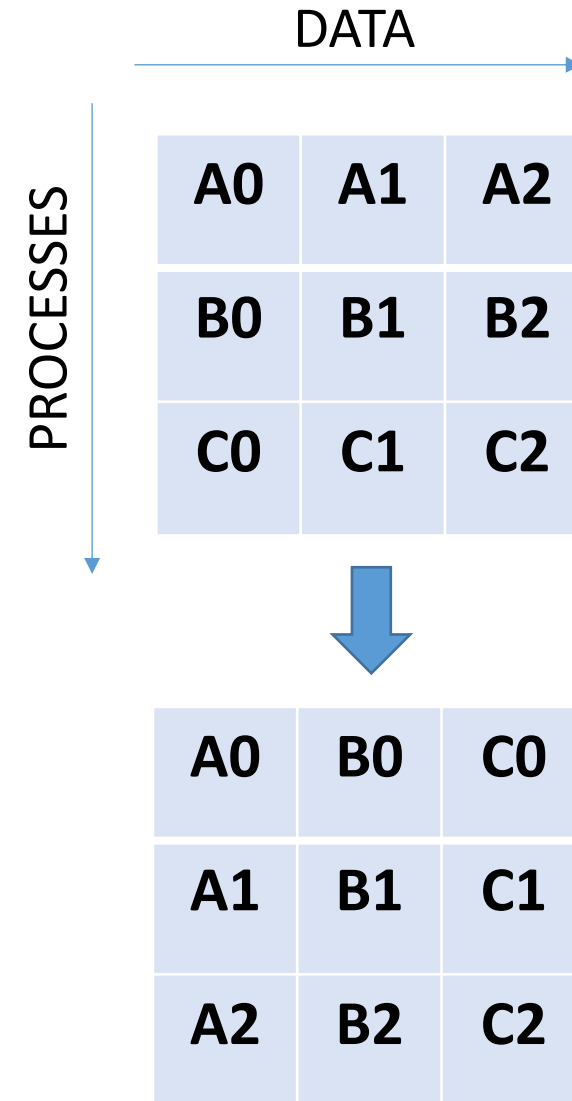
- `MPI_Reduce` followed by `MPI_Bcast`

Alltoall

- Send data from all processes to all processes
- int `MPI_Alltoall` (sendbuf, sendcount, sendtype, *recvbuf*, recvcount, recvttype, comm)
- Output parameter – *recvbuf*
- sendcount/recvcount – sent/received from each process

Equivalent collective?

- `MPI_Scatter` from every process



Homework

1. Broadcast N doubles from rank 0 to all ranks. You can run on any number of processes. $N = 10^3, 10^4, 10^5, 10^6, 10^7$. Study the scalability.
2. Let total number of processes be P . Compare performance of MPI_Gather with Send/Recv vs. MPI_Scatter with Send/Recv on 2, 4, 8, 16, 32 processes.

Self-Assessment 2